Assignment 1

```
#!/bin/bash
FILE="addressbook.txt"
while true
do
   echo ""
   echo "----- ADDRESS BOOK -----"
   echo "1. Create Address Book"
   echo "2. View Address Book"
   echo "3. Insert a Record"
   echo "4. Delete a Record"
   echo "5. Modify a Record"
   echo "6. Exit"
   read -p "Enter your choice: " ch

   case $ch in
      1)
         touch "$FILE"
         echo "Address book created successfully."
         ;;
      2)
         if [ -f "$FILE" ]; then
            if [ -s "$FILE" ]; then
               cat "$FILE"
            else
               echo "Address book is empty."
            fi
         else
            echo "Address book does not exist."
         fi
         ;;
      3)
```

```
    read -p "Enter ID: " id
    read -p "Enter Name: " name
    read -p "Enter Phone: " phone
    read -p "Enter Address: " address

    echo "$id|$name|$phone|$address" >> "$FILE"
    echo "Record inserted successfully."
    ;;
4)
    read -p "Enter ID to delete: " id

    if [ -f "$FILE" ]; then
        grep -v "^$id|" "$FILE" > temp.txt
        if grep -q "^$id|" "$FILE"; then
            mv temp.txt "$FILE"
            echo "Record deleted successfully."
        else
            rm -f temp.txt
            echo "Record not found."
        fi
    else
        echo "Address book does not exist."
    fi
    ;;
5)
    read -p "Enter ID to modify: " id

    if [ -f "$FILE" ] && grep -q "^$id|" "$FILE"; then
        grep -v "^$id|" "$FILE" > temp.txt

        read -p "Enter new Name: " name
        read -p "Enter new Phone: " phone
```

```
            read -p "Enter new Address: " address

            echo "$id|$name|$phone|$address" >> temp.txt
            mv temp.txt "$FILE"
            echo "Record modified successfully."
        else
            echo "Record not found."
        fi
        ;;
    6)
        echo "Exiting Address Book."
        exit 0
        ;;
    *)
        echo "Invalid choice."
        ;;
  esac
done
```

Use below command to run->

chmod +x addressbook.sh

./addressbook.sh

Assignment 2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void sortArray(int arr[], int n)
{
   int i, j, temp;

   for (i = 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (arr[j] > arr[j + 1])
         {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}

int binarySearch(int arr[], int n, int key)
{
   int low = 0;
   int high = n - 1;

   while (low <= high)
   {
```

```
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;

        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int n, i, key, result;

    printf("Enter the number of integers: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of integers.\n");
        return 1;
    }

    int arr[n];

    printf("Enter %d integers:\n", n);

    for (i = 0; i < n; i++)
```

```
    scanf("%d", &arr[i]);

printf("\nParent process started\n");
printf("Parent Process ID: %d\n", getpid());

pid_t pid = fork();

if (pid < 0)
{
    printf("Fork failed.\n");
    return 1;
}

// PARENT PROCESS
if (pid > 0)
{
    printf("Child Process ID: %d\n", pid);

    printf("Parent process sorting the integers...\n");

    sortArray(arr, n);

    printf("Parent sorted array: ");

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n\nParent process waiting for child process...\n");

    wait(NULL);

    printf("Child process completed.\n");
```

```
printf("Parent process resumed after waiting for child.\n");

// ZOMBIE PROCESS
printf("\nDemonstrating Zombie State:\n");

pid_t zombie_pid = fork();

if (zombie_pid == 0)
{
    printf("Child process terminated.\n");
    exit(0);
}
else if (zombie_pid > 0)
{
    printf("Child process created with PID: %d\n", zombie_pid);

    sleep(2);

    printf("Parent has not called wait(), so child may be in ZOMBIE state.\n");
    printf("Zombie Process ID: %d\n", zombie_pid);

    printf("Parent process calling wait()...\n");

    waitpid(zombie_pid, NULL, 0);

    printf("Zombie process removed.\n");
}

// ORPHAN PROCESS
printf("\nDemonstrating Orphan State:\n");

pid_t orphan_pid = fork();
```

```
    if (orphan_pid == 0)
    {
        sleep(2);

        printf("Child process is now an ORPHAN.\n");
        printf("Child Process ID: %d\n", getpid());
        printf("New Parent Process ID: %d\n", getppid());

        printf("Orphan process completed.\n");
        exit(0);
    }
    else if (orphan_pid > 0)
    {
        printf("Child process created with PID: %d\n", orphan_pid);
        printf("Parent process terminating before child.\n");
        exit(0);
    }

    printf("\nParent process completed.\n");
}

// CHILD PROCESS
else
{
    sleep(1);

    printf("\nChild process started\n");
    printf("Child Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());

    sortArray(arr, n);
```

```
        printf("\nChild process performing Binary Search\n");

        printf("Enter element to search: ");
        scanf("%d", &key);

        result = binarySearch(arr, n, key);

        if (result != -1)
            printf("Element %d found at position %d\n", key, result + 1);
        else
            printf("Element %d not found in the array.\n", key);

        printf("\nChild process completed.\n");

        exit(0);
    }

    return 0;
}
```

Assignment 3

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in_index = 0;
int out_index = 0;

pthread_mutex_t mutex;
sem_t empty;
sem_t full;

void *producer(void *arg) {
    int id = *((int *)arg);

    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = (id * 100) + i;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in_index] = item;
```

```
        printf("Producer %d: Inserted item %d at index %d\n",
               id, item, in_index);

        in_index = (in_index + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }

    pthread_exit(NULL);
}

void *consumer(void *arg) {
    int id = *((int *)arg);

    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out_index];
        printf("Consumer %d: Removed item %d from index %d\n",
               id, item, out_index);

        out_index = (out_index + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
```

```
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t prod_threads[NUM_PRODUCERS];
    pthread_t cons_threads[NUM_CONSUMERS];

    int thread_ids[2] = {1, 2};

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pthread_create(&prod_threads[i], NULL,
                       producer, &thread_ids[i]);
    }

    for (int i = 0; i < NUM_CONSUMERS; i++) {
        pthread_create(&cons_threads[i], NULL,
                       consumer, &thread_ids[i]);
    }

    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pthread_join(prod_threads[i], NULL);
    }

    for (int i = 0; i < NUM_CONSUMERS; i++) {
        pthread_join(cons_threads[i], NULL);
```

```
    }

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);

    printf("Main thread exiting.\n");

    return 0;
}
```



```
//Ass-4
//hold&wait,nopreemption, circular, mutual exclusion
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <semaphore.h>

sem_t ch[5];

void *philosopher(void *arg){
    int i = *(int *)arg;
    int left, right;
    printf("\n%d philosopher is created\n", i);
    left = i;
    right = (i + 1) % 5;

    printf("\n%d philosopher is hungry\n", i);

    sem_wait(&ch[left]);
    sem_wait(&ch[right]);

    printf("\n%d philosopher is eating\n", i);

    sleep(5);

    sem_post(&ch[right]);
    sem_post(&ch[left]);
    printf("\n%d philosopher is complete\n", i);

    return NULL;
```

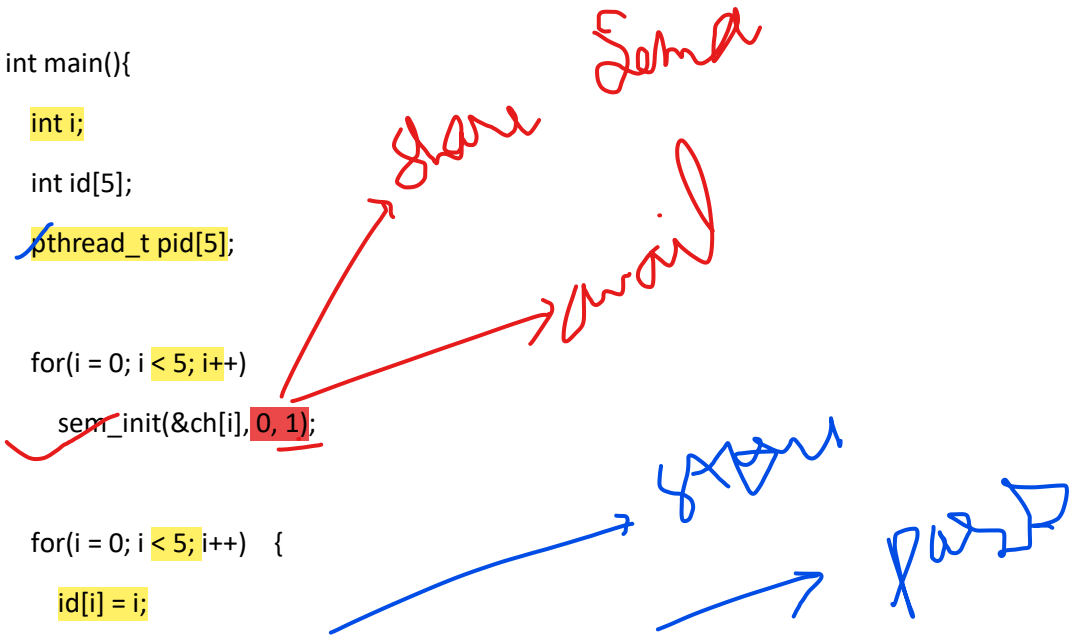
```
}


int main(){
   int i;
   int id[5];
   pthread_t pid[5];


   for(i = 0; i < 5; i++)
      sem_init(&ch[i], 0, 1);


   for(i = 0; i < 5; i++)    {
      id[i] = i;
      pthread_create(&pid[i], NULL, philosopher, &id[i]);
      sleep(i);
   }


   for(i = 0; i < 5; i++)
      pthread_join(pid[i], NULL);
   return 0;
}
```



//ass-5

Assignment 5

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;
   int alloc[10][10], max[10][10];
   int avail[10], need[10][10];
   int finish[10] = {0}, safeSeq[10];
   int count = 0, found;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   printf("Enter Allocation Matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);

   printf("Enter Maximum Requirement Matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < m; j++)
         scanf("%d", &max[i][j]);

   printf("Enter Available Resources:\n");
```

```
for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);

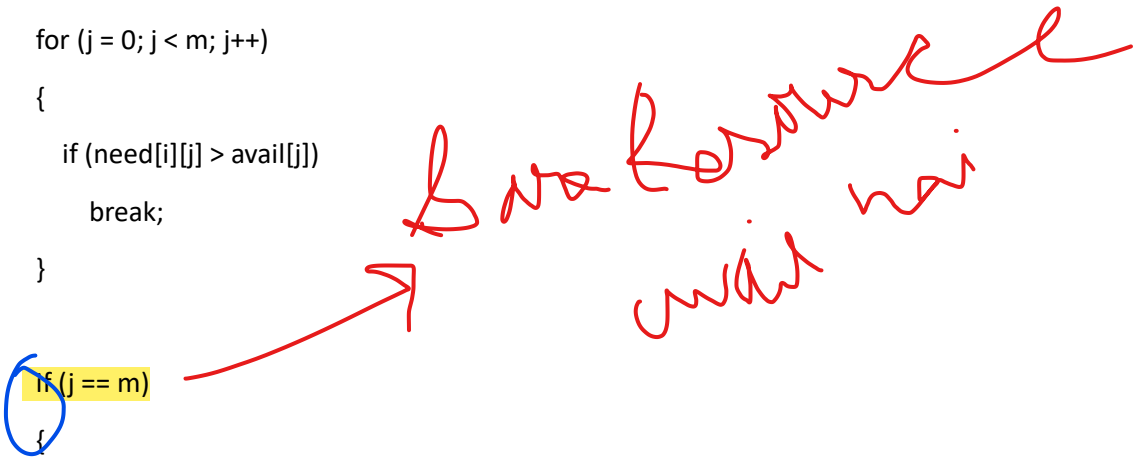
// Calculate Need Matrix
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];

// Banker's Safety Algorithm
while (count < n)
{
    found = 0;

    for (i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            for (j = 0; j < m; j++)
            {
                if (need[i][j] > avail[j])
                    break;
            }

            if (j == m)
            {
                for (k = 0; k < m; k++)
                    avail[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
```



```
            }
        }

    if (found == 0)
    {
        printf("\nSystem is in an UNSAFE state.\n");
        return 0;
    }
  }

  printf("\nNeed Matrix:\n");
  for (i = 0; i < n; i++)
  {
     for (j = 0; j < m; j++)
        printf("%d ", need[i][j]);
     printf("\n");
  }

  printf("\nSystem is in a SAFE state.\n");
  printf("Safe Sequence: ");

  for (i = 0; i < n; i++)
     printf("P%d ", safeSeq[i]);

  printf("\n");

  return 0;
}
```

Assignment 6

Reader :

```
#include<stdio.h>
#include<string.h>
#include<fcntl.h>
#include<unistd.h>
#define FIFO_PIPE "xyz"
int main()
{
   int fd,i,c=0,w=0,l=0;
   char str[100];
   fd = open(FIFO_PIPE,O_RDONLY);
   read(fd,str,sizeof(str));
   for(i=0;str[i]!='\0';i++)
   {
      if(str[i] == ' '|| str[i] == '\n')
      w++;
      if(str[i] == '\n')
      l++;
   c++;
   }
printf("\n No of characters = %d",c);
printf("\n No of lines = %d", l+1);
printf("\n No of words = %d \n",w);
close(fd);
return 0;
}
```

Writer :

```
#include<sys/types.h>
#include<sys/stat.h>
#include<stdio.h>
#include<string.h>
#include<fcntl.h>
#include<unistd.h>
#define FIFO_PIPE "xyz"
int main()
{
   int fd,i=0;
   char str[200], ch;
   mkfifo(FIFO_PIPE,0644);//Pipe Created
   printf("\n Pipe Successfully created \n");
   printf("\n Enter your message(press @ to terminate) \n");
   while((ch = fgetc(stdin)) != '@')
   {
      str[i++] = ch;
   }
   str[i] = '\0';//Null character
   fd = open(FIFO_PIPE,O_WRONLY);
   write(fd,str,strlen(str));
   close(fd);
   return 0;
}
```

Assignment 7

```
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
   int fd[2], n;
   char buffer[100];
   pid_t p;

   pipe(fd);
   p = fork();

   if (p > 0){
      printf("Passing value to child\n");
      write(fd[1], "hello\n", 6);
   }else{
      printf("Child received data\n");
      n = read(fd[0], buffer, 100);
      write(1, buffer, n);
   }
   return 0;
}
```

Assignment 8

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int RQ[100], n, initial, size;
int dataEntered = 0;

void getData()
{
   int i;

   printf("Enter the number of requests: ");
   if (scanf("%d", &n) != 1 || n < 1 || n > 100)
   {
      printf("Invalid number of requests\n");
      dataEntered = 0;
      return;
   }

   printf("Enter total disk size: ");
   if (scanf("%d", &size) != 1 || size <= 0)
   {
      printf("Invalid disk size\n");
      dataEntered = 0;
      return;
   }

   printf("Enter the request sequence:\n");
```

```
    for (i = 0; i < n; i++)
    {
        if (scanf("%d", &RQ[i]) != 1 ||
            RQ[i] < 0 || RQ[i] >= size)
        {
            printf("Invalid disk request\n");
            dataEntered = 0;
            return;
        }
    }

    printf("Enter initial head position: ");
    if (scanf("%d", &initial) != 1 ||
        initial < 0 || initial >= size)
    {
        printf("Invalid initial head position\n");
        dataEntered = 0;
        return;
    }

    dataEntered = 1;
}

void sstf()
{
    int visited[100] = {0};
    int current = initial;
    int i, j, index, min, d;
    int TotalHeadMoment = 0;

    printf("\nSSTF Head Movement: %d", current);
```

```
    for (i = 0; i < n; i++)
    {
        min = INT_MAX;
        index = -1;

        for (j = 0; j < n; j++)
        {
            if (visited[j] == 0)
            {
                d = abs(RQ[j] - current);

                if (d < min)
                {
                    min = d;
                    index = j;
                }
            }
        }

        TotalHeadMoment += min;
        current = RQ[index];
        visited[index] = 1;

        printf(" -> %d", current);
    }

    printf("\nTotal head movement is %d\n",
           TotalHeadMoment);
}

void sort(int arr[])
{
```

```
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void scan()
{
    int arr[100], i, index = n;
    int current = initial;
    int TotalHeadMoment = 0;

    for (i = 0; i < n; i++)
        arr[i] = RQ[i];

    sort(arr);

    for (i = 0; i < n; i++)
    {
        if (arr[i] >= initial)
        {
            index = i;
```

```
            break;
        }
    }

    printf("\nSCAN Head Movement: %d", current);

    // Move towards higher cylinder numbers
    for (i = index; i < n; i++)
    {
        TotalHeadMoment += abs(arr[i] - current);
        current = arr[i];
        printf(" -> %d", current);
    }

    // Move to last cylinder
    TotalHeadMoment += abs((size - 1) - current);
    current = size - 1;
    printf(" -> %d", current);

    // Reverse direction
    for (i = index - 1; i >= 0; i--)
    {
        TotalHeadMoment += abs(arr[i] - current);
        current = arr[i];
        printf(" -> %d", current);
    }

    printf("\nTotal head movement is %d\n",
           TotalHeadMoment);
}

void clook()
```

```
{
    int arr[100], i, index = n;
    int current = initial;
    int TotalHeadMoment = 0;

    for (i = 0; i < n; i++)
        arr[i] = RQ[i];

    sort(arr);

    for (i = 0; i < n; i++)
    {
        if (arr[i] >= initial)
        {
            index = i;
            break;
        }
    }

    printf("\nC-LOOK Head Movement: %d", current);

    // Move towards higher requests
    for (i = index; i < n; i++)
    {
        TotalHeadMoment += abs(arr[i] - current);
        current = arr[i];
        printf(" -> %d", current);
    }

    // Jump to lowest request and move upwards
    for (i = 0; i < index; i++)
    {
```

```
        TotalHeadMoment += abs(arr[i] - current);
        current = arr[i];
        printf(" -> %d", current);
    }

    printf("\nTotal head movement is %d\n",
          TotalHeadMoment);
}

int main()
{
    int choice;
    while (1)
    {
        printf("\n\nDisk Scheduling Algorithms\n");
        printf("1. Enter Data\n");
        printf("2. SSTF\n");
        printf("3. SCAN\n");
        printf("4. C-LOOK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1)
            return 0;
        if (choice == 1)
        {
            getData();
        }
        else if (choice >= 2 && choice <= 4){
            if (!dataEntered)
            {
                printf("Please enter data first!\n");
```

```
            continue;
        }

        switch (choice)
        {
            case 2:
                sstf();
                break;
            case 3:
                scan();
                break;
            case 4:
                clook();
                break;
        }
    }
    else if (choice == 5){
        printf("Exiting program...\n");
        return 0;
    }else{
        printf("Invalid choice!\n");
    }
  }
}
```